BACKEND DEVELOPER 1 - TASK LIST

Focus: Database, API Core, Email Processing

ID	Day	Task	Description	Effort (hrs)
BE1-01	Day 1	Database Schema	Create work_orders table with all required fields	2
BE1-02	Day 1	API Project Setup	Initialize FastAPI project, folder structure, dependencies	1
BE1-03	Day 1	Database Models	Create SQLAlchemy models for work_orders	1.5
BE1-04	Day 1	Email Parser POC - Hard code email body	Build email parsing function with sample data	2
BE1-05	Day 1	Basic CRUD Endpoints	GET, POST work orders endpoints	2
BE1-06	Day 2	AI Extraction Service	openai API integration for all extraction extraction	3
BE1-07	Day 2	Work Order Flow	work order creation	2
BE1-08	Day 2	Status Update Endpoint	PATCH /work-orders/{id}/status	1
BE1-09	Day 2	Search & Filter	Add query params to list endpoint	2
BE1-10	Day 3	Approval Endpoint	POST /work-orders/{id}/approve	2
BE1-11	Day 3	Mock CMMS Integration	Stub CMMS send function	1.5
BE1-12	Day 3	Validation Layer	Add input validation and error handling	2
BE1-13	Day 4	Error Handling	Comprehensive error handling across all endpoints	2
BE1-14	Day 4	API Documentation	Auto-generate OpenAPI docs	1
BE1-15	Day 4	Integration Testing	Test email → creation → approval flow	2
BE1-16	Day 5	Bug Fixes	Fix issues from Day 4 testing	3

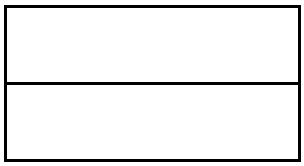
BE1-17	Day 5	Performance Check	Check query performance, add indexes	1
BE1-18	Day 5	Deployment Prep	Docker setup, environment configs	2

TOTAL ES **33**

Dependencies	Status	Claude Code Prompt	Priority
None	Not Started	Create PostgreSQL schema for work_orders table with fields: work_order_id, source, asset, location, issue_description, priority, status, created_at, etc.	Critical
None	Not Started	Set up FastAPI project with folder structure: /api, /models, /services, /db. Include requirements.txt with fastapi, uvicorn, sqlalchemy, anthropic	Critical
BE1-01	Not Started	Create SQLAlchemy ORM models for WorkOrder with all fields matching schema	Critical
None	Not Started	Create email parser that extracts subject, body, from, and stores in dict. Test with sample email	High
BE1-03	Not Started	Create FastAPI endpoints: POST /work-orders, GET /work-orders, GET /work-orders/{id}	Critical
BE1-04	Not Started	Create service class that calls Claude API to extract asset, location, issue from email/workspace query(AI Assessment, Safety Detection ,Compliance , Location,Intelligence ,Clearance ,Parts List ,Inventory ,Vendors ,Technician ,Schedule ,Workspace Pinned with quick actions,Journey). Return structured JSON	Critical
BE1-06	Not Started	Integrate email parser + AI extraction + work order creation into single flow	Critical
BE1-05	Not Started	Create endpoint to update work order status with validation	High
BE1-05	Not Started	Add filters to GET /work-orders: status, priority, asset, date range	Medium
BE1-05	Not Started	Create approval endpoint that changes status and records approver	High
BE1-05	Not Started	Create mock CMMS service that logs work order and returns success	Medium
BE1-05	Not Started	Add Pydantic models for request validation and proper error responses	High
BE1-12	Not Started	Add try-catch blocks, proper HTTP status codes, error messages	High
BE1-13	Not Started	Ensure FastAPI auto-docs are complete with descriptions	Medium
BE1-07, BE1-10	Not Started	Create test cases for complete workflow using pytest	High
BE1-15	Not Started	Address all critical bugs found in testing	Critical

BE1-16	Not Started	Run EXPLAIN on queries, add missing indexes	Medium
BE1-16	Not Started	Create Dockerfile, docker-compose.yml for easy deployment	High

Notes
Use provided schema
Mock Outlook integration
Use simple prompt
Can be stubbed



BACKEND DEVELOPER 2 - TASK LIST

Focus: Journey Tracking, Asset Management, Supporting Services

ID	Day	Task	Description	Effort (hrs)
BE2-01	Day 1	Journey Log Schema	Create journey_logs table	2
BE2-02	Day 1	Asset Reference Table	Create assets table for lookups	1.5
BE2-03	Day 1	Journey Log Models	SQLAlchemy models for journey logs	1.5
BE2-04	Day 1	Asset Lookup Endpoint	GET /assets, GET /assets/{id}	2
BE2-05	Day 1	Sample Data Seeding	Create seed data script	1.5
BE2-06	Day 2	Journey Creation Service	Auto-create journey on work order creation	3
BE2-07	Day 2	Journey Endpoints	GET /journeys, GET /journeys/{id}	2.5
BE2-08	Day 2	Milestone Update	PATCH /journeys/{id}/milestone	2.5
BE2-09	Day 3	Work Order History	GET /work-orders/{id}/history	2
BE2-10	Day 3	Dashboard Stats	GET /dashboard/stats	2.5
BE2-11	Day 3	Location Endpoints	GET /locations for dropdown	2.5
BE2-12	Day 4	Bulk Status Update	PATCH /work-orders/bulk/status	2.5
BE2-13	Day 4	Journey Analytics	GET /journeys/analytics	2.5
BE2-14	Day 4	Integration Support	Help with integration Shashank code	3
BE2-15	Day 5	Bug Fixes	Fix issues from testing	3
BE2-16	Day 5	API Performance	Optimize slow queries	1.5
BE2-17	Day 5	Documentation	README and API docs	2

TOTAL ES **38**

Dependencies	Status	Claude Code Prompt	Priority
BE1-01	Not Started	Create PostgreSQL schema for journey_logs with fields: jlog_id, work_order_id, status, milestones (JSON), expected_timeline (JSON)	Critical
None	Not Started	Create assets table with: asset_id, asset_name, type, location, manufacturer, model	High
BE2-01	Not Started	Create JourneyLog ORM model with relationships to WorkOrder	Critical
BE2-02	Not Started	Create endpoints to search and retrieve asset information	High
BE2-02	Not Started	Create Python script to populate sample assets, locations for testing	Medium
BE2-03, BE1-05	Not Started	Create service that automatically creates journey log when work order is created	Critical
BE2-06	Not Started	Create endpoints to retrieve journey logs	High
BE2-07	Not Started	Endpoint to update milestone status in journey	Medium
BE1-05	Not Started	Endpoint to get status change history for work order	Medium
BE1-05	Not Started	Endpoint returning counts by status, priority, asset type	High
BE2-02	Not Started	Create endpoints to list locations for forms	Medium
BE1-08	Not Started	Endpoint to update multiple work orders at once	Low
BE2-07	Not Started	Return journey completion rates, average times	Low
BE2-06	Not Started	Work with BE1 on end-to-end testing	High
BE2-14	Not Started	Address bugs in journey and asset services	Critical
BE2-15	Not Started	Add pagination, optimize joins	Medium
BE2-15	Not Started	Write setup instructions and API usage guide	High

Notes
Use provided schema
Sample data included
Helps frontend dev
Audit trail
For frontend dashboard
Nice to have
Phase 2 feature
Collaborative

FRONTEND DEVELOPER - TASK LIST

Focus: React UI, API Integration, User Experience

ID	Day	Task	Description	Effort (hrs)
FE-01	Day 1	React Project Setup	Create React app with routing, state management	2
FE-02	Day 1	API Client Setup	Configure axios with base URL and interceptors	1
FE-03	Day 1	Layout Components	Header, sidebar, main layout	2
FE-04	Day 1	Mock Data	Create mock API responses for development	1.5
FE-05	Day 2	Work Order List Page	Table view with filters	4
FE-06	Day 2	List Item Component	Reusable work order card	1.5
FE-07	Day 2	Status Badge Component	Visual status indicators	1
FE-08	Day 3	Work Order Detail Page	Full work order view	3
FE-09	Day 3	Create Work Order Form	Form for manual creation	3
FE-10	Day 3	Approval Action Button	Button to approve work orders	1.5
FE-11	Day 4	Status Update Modal	Dialog to change work order status	2
FE-12	Day 4	Journey Timeline View	Visual journey progress	2.5
FE-13	Day 4	Dashboard Page	Stats and charts overview	2.5
FE-14	Day 4	Loading States	Spinners and skeletons	1.5
FE-15	Day 4	Error Handling	Error messages and retry	1.5
FE-16	Day 5	Polish & Refinement	UI/UX improvements	3
FE-17	Day 5	End-to-End Testing	User flow testing	2
FE-18	Day 5	Demo Preparation	Clean up, add sample data	2

TOTAL ES **37.5**

Dependencies	Status	Claude Code Prompt
None	Not Started	Create React app with: react-router-dom, axios, react-query, tailwind. Set up folder structure /components, /pages, /api, /hooks
FE-01	Not Started	Set up axios instance with base URL, error interceptors, auth headers
FE-01	Not Started	Create AppLayout with navigation sidebar and header
None	Not Started	Create mock work order data for testing before backend is ready
FE-02, BE1-05	Not Started	Create WorkOrderList component with table showing: ID, asset, status, priority, date. Add status and priority filters
FE-05	Not Started	Create WorkOrderCard component for list items
FE-05	Not Started	Create StatusBadge with color coding for different statuses
FE-02, BE1-05	Not Started	Create WorkOrderDetail page showing all fields, with edit capability
FE-02, BE1-05, BE2-04	Not Started	Create form with: asset dropdown, location, issue description, priority. Validate on submit
FE-08, BE1-10	Not Started	Add Approve button on detail page that calls approval endpoint
FE-08, BE1-08	Not Started	Create modal with status dropdown and notes field
FE-08, BE2-07	Not Started	Create timeline component showing journey milestones
BE2-10	Not Started	Create dashboard with counts by status, priority pie chart
All pages	Not Started	Add loading indicators to all data fetching
All pages	Not Started	Add error boundaries and user-friendly error messages
All	Not Started	Improve spacing, alignment, colors, responsiveness
All	Not Started	Test complete workflows: create, view, approve, update
All	Not Started	Ensure demo environment works smoothly with good data

Priority	Notes
Critical	Use Vite for speed
Critical	
High	Use Tailwind
High	Unblocks development
Critical	Wait for BE API
High	
Medium	
Critical	
Critical	
High	
High	
Medium	Nice to have
Medium	
High	Better UX
High	
High	
Critical	
Critical	

CLAUDE CODE - PROMPT LIBRARY

Copy these prompts to Claude Code for faster development

Task	Claude Code Prompt
Database Schema	Create PostgreSQL schema for work_orders table with these fields: - work_order_id (VARCHAR 50, PK) - source (VARCHAR 50) - asset_id, asset_name, location - issue_description (TEXT) - priority (VARCHAR 20) - status (VARCHAR 50) - created_at, updated_at (TIMESTAMP) - criticality, safety, compliance (JSONB) Add appropriate indexes on status, created_at, asset_id
FastAPI Setup	Create FastAPI project structure: /api - API endpoints /models - Pydantic models /db - Database connection /services - Business logic Include main.py with FastAPI app initialization, CORS middleware, and /health endpoint. Add requirements.txt with: fastapi, uvicorn, sqlalchemy, psycopg2, pydantic, anthropic, python-dotenv
WorkOrder Model	Create SQLAlchemy ORM model for WorkOrder: - Map to work_orders table - Include all fields from schema - Add relationship to JourneyLog - Add __repr__ method - Use PostgreSQL JSONB for criticality, safety, compliance fields
CRUD Endpoints	Create FastAPI CRUD endpoints for work orders: POST /api/work-orders - Create work order GET /api/work-orders - List with filters (status, priority, limit, offset) GET /api/work-orders/{id} - Get single work order PATCH /api/work-orders/{id} - Update work order Use Pydantic models for request/response validation. Return proper HTTP status codes. Include error handling.

AI Email Extractor	Create service to extract work order info from email using Claude API: Input: email text (subject + body) Output: JSON with {asset, location, issue_description, priority} Use anthropic SDK with this prompt: 'Extract work order information from this email: {email_text} Return JSON with: asset, location, issue_description, priority (low/medium/high/urgent)' Handle API errors gracefully.
React App Setup	Create React app using Vite: npm create vite@latest work-order-app -- --template react Add dependencies: npm install react-router-dom axios react-query @tanstack/react-query tailwindcss Set up: 1. Tailwind config 2. Router in main.jsx with routes: /, /work-orders, /work-orders/:id 3. API client in api/client.js with axios base URL 4. Folder structure: /components, /pages, /hooks, /api
Work Order List Component	Create WorkOrderList component: - Fetch work orders from GET /api/work-orders - Display in table: ID, Asset, Location, Status, Priority, Date - Add filters: status dropdown, priority dropdown, search by asset - Use react-query for data fetching - Show loading spinner while fetching - Status badge with color coding (pending=yellow, approved=green, completed=blue) - Click row to navigate to detail page - Use Tailwind for styling

Work Order Detail Page	Create WorkOrderDetail page: - Fetch single work order from GET /api/work-orders/{id} - Display all fields in card layout - Show journey timeline if available - Add action buttons: Approve, Update Status, Close - Handle loading and error states - Use Tailwind card styling
Create Work Order Form	Create form to manually create work order: - Fields: asset (dropdown from /api/assets), location, issue description (textarea), priority (dropdown) - Validate required fields - POST to /api/work-orders on submit - Show success message and redirect to list - Use react-hook-form for validation - Tailwind styling
Journey Timeline	Create Journey Timeline component: - Props: journeyLog object - Display milestones as vertical timeline - Color code: completed (green), in progress (yellow), pending (gray) - Show dates for completed milestones - Use Tailwind for timeline styling

Expected Output	Notes
Complete CREATE TABLE statement with indexes	Run in psql or migration tool
Complete project folder structure with files	Use uvicorn for dev server
models/work_order.py with complete ORM model	Import in db/base.py
api/work_orders.py with all endpoints	Test with curl or Postman

services/email_extractor.py with extract_work_order(email_text) function	Set ANTHROPIC_API_KEY in .env
Complete React project setup	Start with npm run dev
components/WorkOrderList.jsx	Will need StatusBadge component

pages/WorkOrderDetail.jsx	Add to router
components/CreateWorkOrderForm.jsx	Add to /create route
components/JourneyTimeline.jsx	Shows on detail page

MVP SCOPE DEFINITION

IN SCOPE (MVP - Must Have by Friday)
✓ Email-based work order creation
✓ AI extraction of asset, location, issue
✓ Work order list view with filters
✓ Work order detail view
✓ Manual work order creation form
✓ Basic approval workflow (approve button)
✓ Status update functionality
✓ Journey log creation (auto on WO create)
✓ Basic dashboard with counts
✓ Asset and location lookup
✓ Mock CMMS integration

RATIONALE

5-day sprint requires focus on core functionality
MVP proves concept and enables user feedback
Complex features (AI assessment, scheduling) need m
Phase 2 can build on working foundation

DEMO FOCUS

1. Show email → work order creation
2. Display work order list with real data
3. View work order details
4. Approve a work order
5. Update status
6. Show journey log

ore time